

Лабораторная работа № 4 по курсу дискретного анализа: строковые алгоритмы

Выполнил студент группы М8О-208Б-22 МАИ *Немкова Анастасия*.

Условие

Необходимо реализовать один из стандартных алгоритмов поиска образцов для указанного алфавита.

Вариант алгоритма: Поиск одного образца-маски: в образце может встречаться «джокер» (представляется символом ? — знак вопроса), равный любому другому символу. При реализации следует разбить образец на несколько, не содержащих «джокеров», найти все вхождения при помощи алгоритма Ахо-Корасик и проверить их относительное месторасположение.

Вариант алфавита: Слова не более 16 знаков латинского алфавита (регистронезависимые)

Метод решения

Алгоритм Ахо-корасик - это алгоритм поиска подстроки в строке, который реализует поиск множества подстрок из словаря в данной строке.

Для начала из всех подстрок образца, не содержащих джокеров, строим trie - дерево, содержащее корень, равный пустой строке, рёбра, по которым можно перейти к следующей вершине по символу, и другие вершины, являющиеся префиксами образца.

Затем для каждой вершины данного дерева строим связи неудач, обходя дерево в ширину. Связь неудач связывает две вершины дерева - первая, которая получена в результате прохода по тексту и совпадения символов текста с образцом до вершины, в которой найдено несовпадение; вторая - вершина другого образца, префикс которого является максимальным суффиксом пройденного пути. Для корня и вершин первого уровня суффиксной ссылкой будет ссылка на корень. Для любой другой вершины нужно перейти по суффиксной ссылке её родителя в узел n . Если из узла n можно перейти в другой узел x по последнему символу строки исходной вершины, тогда ставим суффиксную ссылку на узел x . Если нельзя перейти, тогда переходим по суффиксной ссылке узла n , пока не упрёмся в корень и делаем то же самое. Если дошли до корня, тогда ставим суффиксную ссылку на корень. Также при построении дерева нужно найти длины подстрок образца, не содержащих джокеров, и их начальные позиции в образце для дальнейшей проверки местоположения подстрок.

При поиске необходимо пройти по тексту и найти вхождения каждого подобразца в текст. Для каждого такого начального вхождения мы увеличиваем счетчик в ячейке [текущая позиция в тексте - позиция подобразца в образце] вектора, равного длине тек-

ста и инициализированного нулями, на единицу. В итоге образец считается найденным в тексте, если на i -ой позиции данного вектора находится значение, равное количеству подобразцов без джокеров.

Описание программы

Для реализации словаря был создан класс дерева Trie, каждый узел которого содержит ассоциативный массив для хранения дочерних узлов текущего узла, Указатель на узел, к которому происходит переход в случае неудачного совпадения текущего символа, и вектор индексов подобразцов, которые заканчиваются в данном узле. Само дерево хранит указатель на корень, вектор с длинами подобразцов и счетчик подобразцов без джокеров. В классе Trie реализованы следующие методы:

- `void Create(const std::vector<std::string>& patterns)` - построение дерева на основе входного образца.
- `void Search(const std::vector<std::string>& text, const int& patSize, std::vector<std::pair<int, int>& answer)` - поиск подобразцов в тексте.
- `void CreateLinks()` - создание ссылок на неудачные переходы.

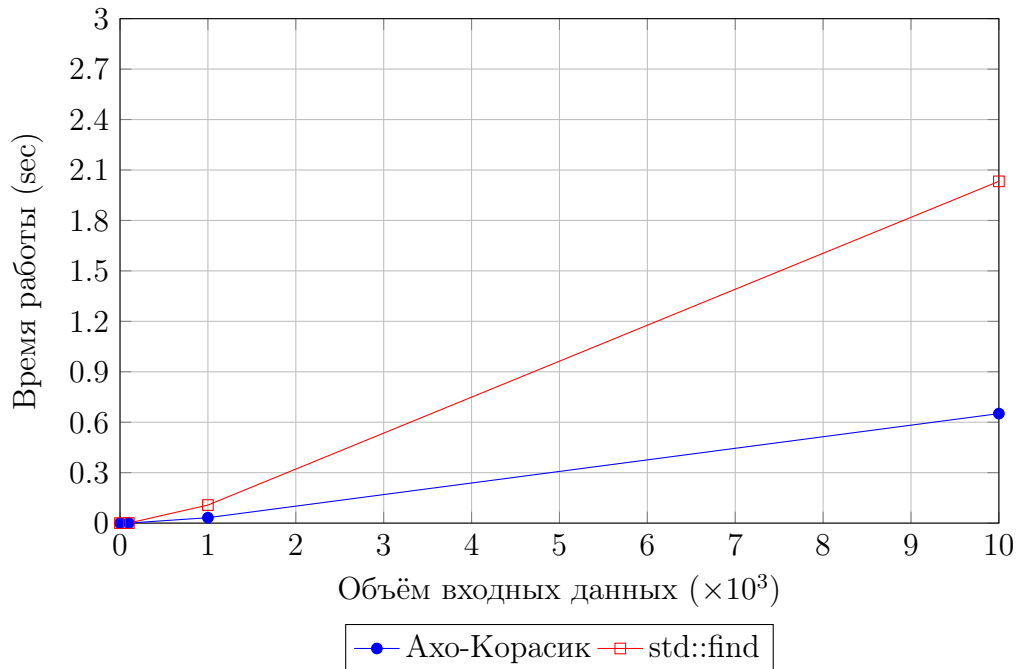
Дневник отладки

1. 23 май 2024, 02:16:59 ОК на 1 тесте

Тест производительности

Для измерения производительности сравнивается время работы реализованного алгоритма Ахо-Корасик и наивного поиска с помощью стандартного контейнера `std::find`. Подсчет времени производился с помощью библиотеки `chrono`, которая позволяет фиксировать время в начале и конце выполнения сортировки. Первый тест состоит из 10^2 строк по 50 слов в каждой и 20 слов в паттрене, второй - из 10^3 строк по 500 слов в каждой и 100 слов в паттрене и третий из 10^4 строк по 1000 слов в каждой и 100 слов в паттрене.

На графике представлена зависимость времени выполнения поиска от объема входных данных.



Выводы

В ходе данной лабораторной работы был изучен алгоритм Ахо-Корасика, позволяющий искать несколько образцов в строке за линейное время. Также была создана структура дерева Trie и реализован автомат для поиска по этому дереву.

Время требуемое на построение дерева — $O(n)$, где n — суммарная длина всех подобразцов, не содержащих джокера. Если число джокеров ограничено фиксированной постоянной, то решение задачи поиска возможно за линейное время $O(n + m)$, где n — суммарная длина всех подстрок образца без джокеров, m — длина текста. Для отображения образца в дерево используется тар, реализованная на красно-черном дереве. Время обращения к элементам пропорционально логарифму числа элементов, а это константа. Время поиска вхождений составляет $O(m + t)$, t — число вхождений. Число джокеров ограничено, независимо от размера образца, поэтому алгоритм работает за линейное время.